

## Object Design

This chapter will describe the packages of the Task Management System (TMS) subsystems with classes, attributes, and methods using UML class diagrams. Overall, four subsystems – **user management**, **project and task management**, **collaboration**, and **notifications subsystems** – will be described. The **data management subsystem** cannot adequately be represented using a UML class diagram because the latter is not able to capture the dynamic behaviour of databases.

### 1. User Management Package

The user management subsystem contains the following classes: `User`, `Role`, `RoleType`, `Permission`, and `AuthenticationService`.

The class `User` represents a task management system user with such attributes as `userID`, `name`, `email`, `password`, `timeZone`, and `role`. Methods include updating and editing their profile.

The class `Role` represents a role with such attributes as `type` and `permissions`. Methods include adding and removing permissions.

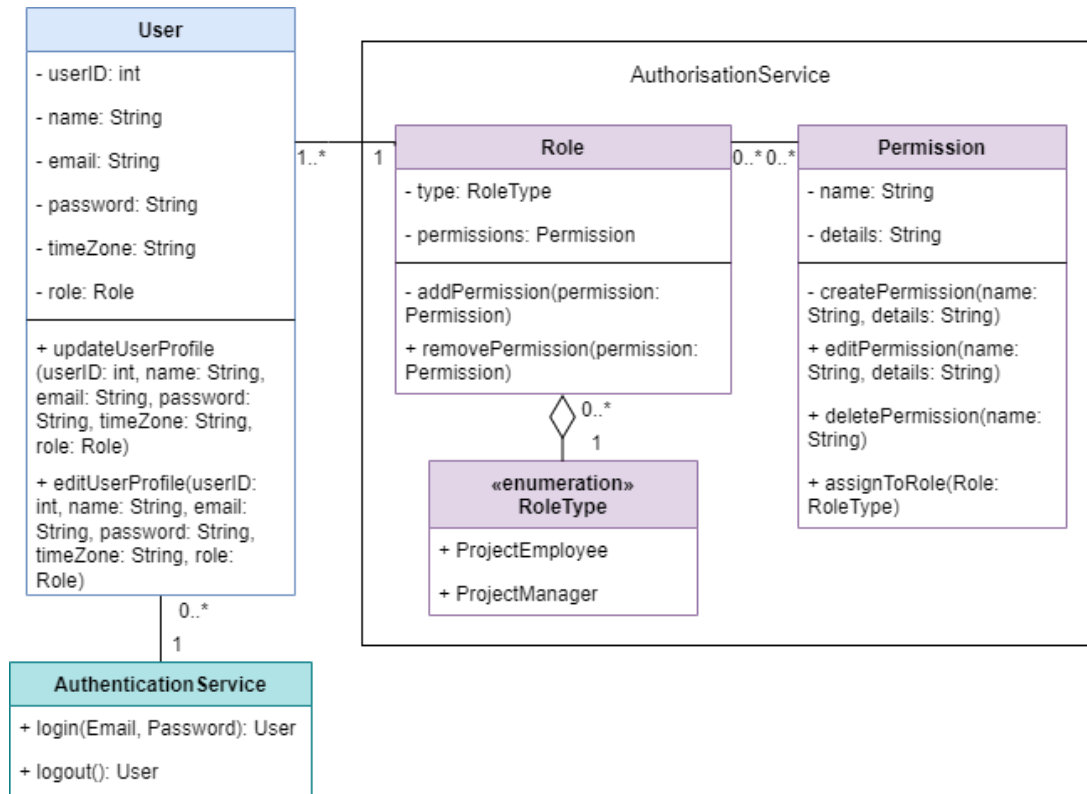
The class `RoleType` represents the `projectEmployee` and `projectManager` enumerations.

The class `Permission` represents permissions assigned to roles with such attributes as `name` and `details`. Methods include creating, editing, and deleting permission and also assigning permission to the role.

The class `AuthenticationService` manages such user authentication processes as login and logout with the respective methods.

**Figure 1**

*The UML Class Diagram of the User Management Package*



Source: Own results.

## 2. Project and Task Management Package

The project and task management subsystem contains the following classes: `Project`, `ProjectStatuses`, `ProjectConditions`, `ProjectPhases`, `ProjectService`, `Task`, `TaskStatuses`, `TaskDescription`, `Subtask`, `TaskService`, `Comment`, `Dependency`, `DependencyType`, `GanttChart`, and `BurnDownChart`.

The class `Project` represents a project with such attributes as `projectID`, `title`, `startDate`, `deadline`, `estimatedDateToFinish`, `status`, `condition`, `phase`, `risk`, `issue`, `damage`, `dependency`, `tasks`, `progressPercentage`, `ganttChart`, and `burnDownChart`. Methods include adding and removing tasks, arranging tasks order, adding task dependencies, generating Gantt and Burndown charts, and calculating and updating progress percentage.

The class `ProjectStatuses` represents the active, completed, active, and onHold enumerations.

The class `ProjectConditions` represents the `toBeStarted`, `onTrack`, `atRisk`, `offTrack`, and `Completed` enumerations.

The class `ProjectPhases` represents the initiating, planning, executing, control, and completed enumerations.

The class `ProjectService` manages requests for creating, updating, and deleting a project with the respective methods.

The class `Task` represents a project's task with such attributes as `taskID`, `projectID`, `title`, `startDate`, `deadline`, `status`, `description`, `assignedTo`, `tag`, `subtasks`, and `comments`. Methods include adding description, assigning tasks, adding subtasks, arranging subtasks order, tagging tasks, adding and removing comments, pinning and unpinning a comment, and repeating tasks.

The class `TaskStatuses` represents the `awaitingAssignment`, `inProgress`, `inReview`, `completed`, `unsuccessful`, `toBeModified` enumerations.

The class `TaskDescription` represents a task description allowing the user to create and update description and attach images, videos, and text files with the respective methods.

The class `Subtask` represents a task subtask with such attributes as `title`, `subtaskID`, and `description`. Methods include `createSubtask`, `updateSubtask`, and `deleteSubtask`.

The class `TaskService` manages requests for creating, updating, and deleting a task with the respective methods.

The class `Comment` represents a task comment with attributes such as `commentID`, `commentContent`, `taskID`, `projectID`, `author`, and `time`.

The class `Dependency` represents a dependent relationship between tasks with a `type` attribute and the `dependsOn` method.

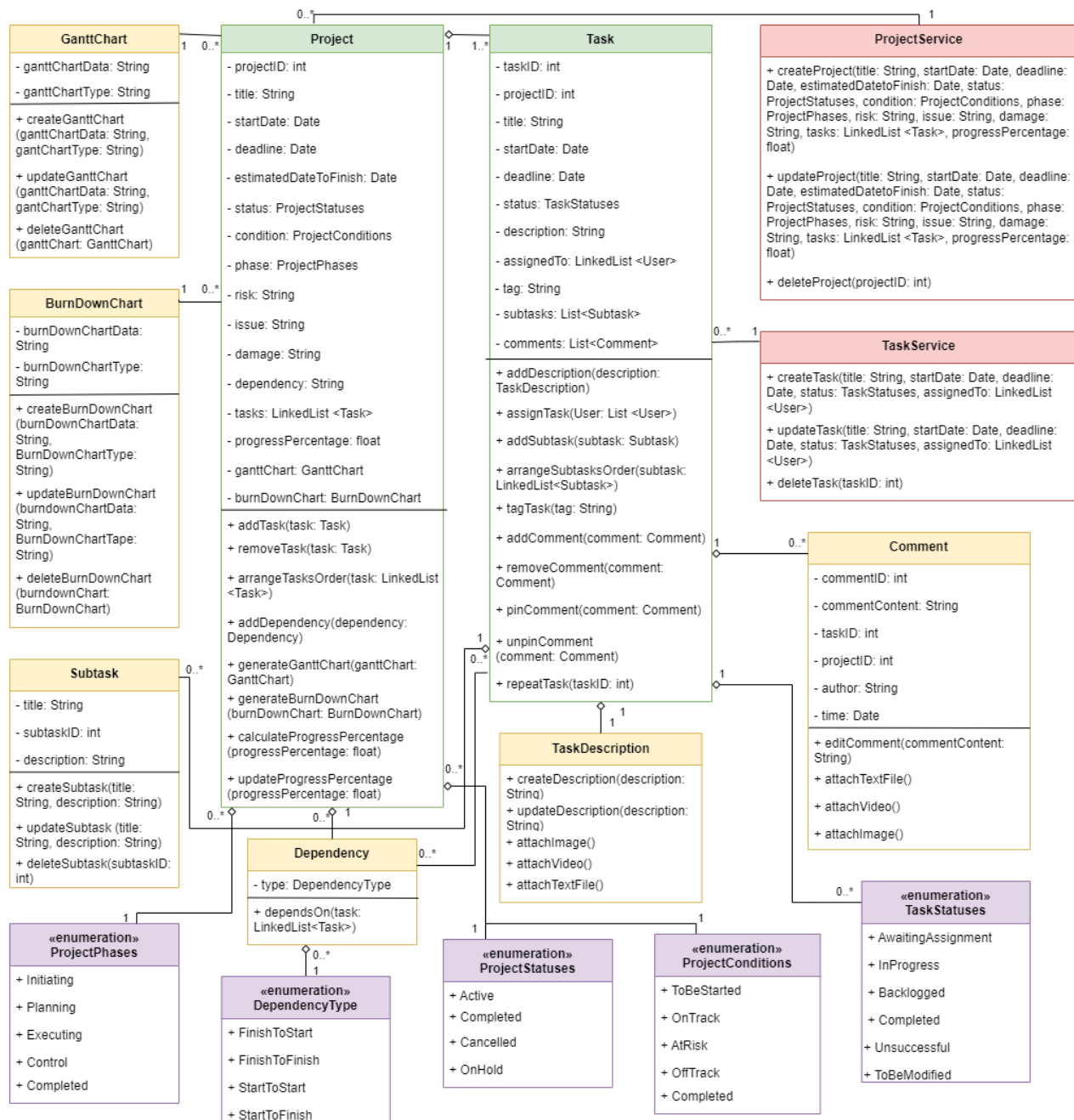
The class `DependencyType` represents the `finishToStart`, `finishToFinish`, `startToStart`, and `startToFinish` enumerations.

The class `GanttChart` represents a project's Gantt chart with such attributes as `gantChartData` and `gantChartType`. Methods include `createGantChart`, `updateGanttChart`, and `deleteGanttChart`.

The class `BurnDownChart` represents a project's Burn Down chart with such attributes as `burnDownChartData` and `burnDownChartType`. Methods include `createBurnDownChart`, `updateBurnDownChart`, and `deleteBurnDownChart`.

**Figure 2**

*The UML Class Diagram of the Project and Task Management Package*



Source: Own results.

### 3. Collaboration Package

The collaboration subsystem contains the following classes: Call, Message, File, CallService, MessageService, and FileService.

The class Call represents an audio or video call with such attributes as callID, startDate, endDate, duration, and participants. Methods include addParticipant and removeParticipant.

The class `Message` represents a message with such attributes as `messageID`, `userID`, `content`, and `timeDelivered`. It contains the `editContent` method.

The class `File` represents an image, video, or text file shared by the user and contains such attributes as `fileID`, `userID`, `fileName`, `fileType`, and `uploadedDate`. It contains the `editFileName` method.

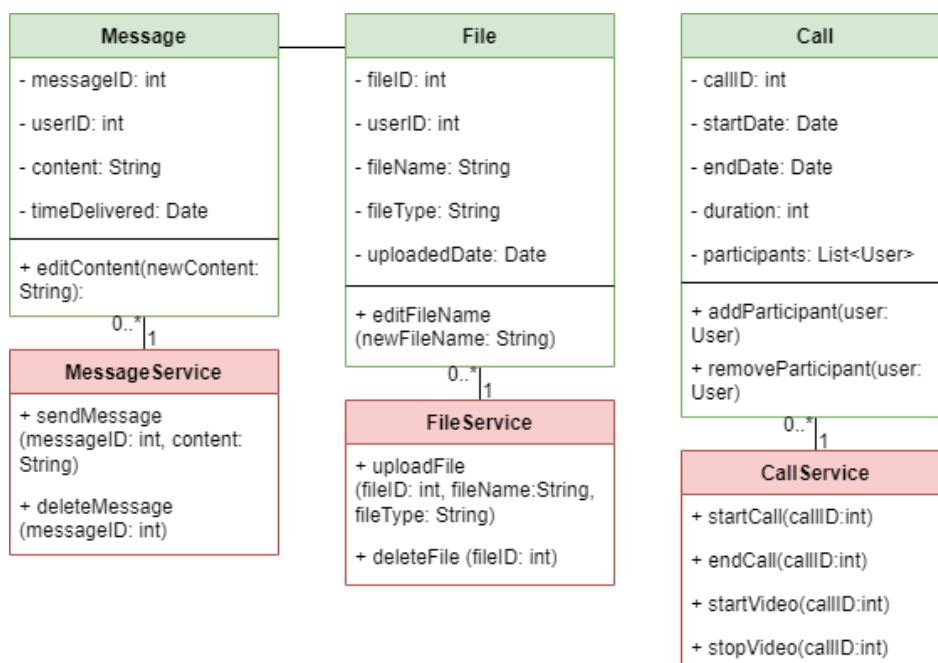
The class `CallService` manages requests for starting and ending a call and starting and stopping video during a call with the respective methods.

The class `MessageService` manages requests for sending and deleting a message with the respective methods.

The class `FileService` manages requests for uploading and deleting a file with the respective methods.

**Figure 3**

*The UML Class Diagram of the Collaboration Package*



Source: Own results.

#### 4. Notification Subsystem

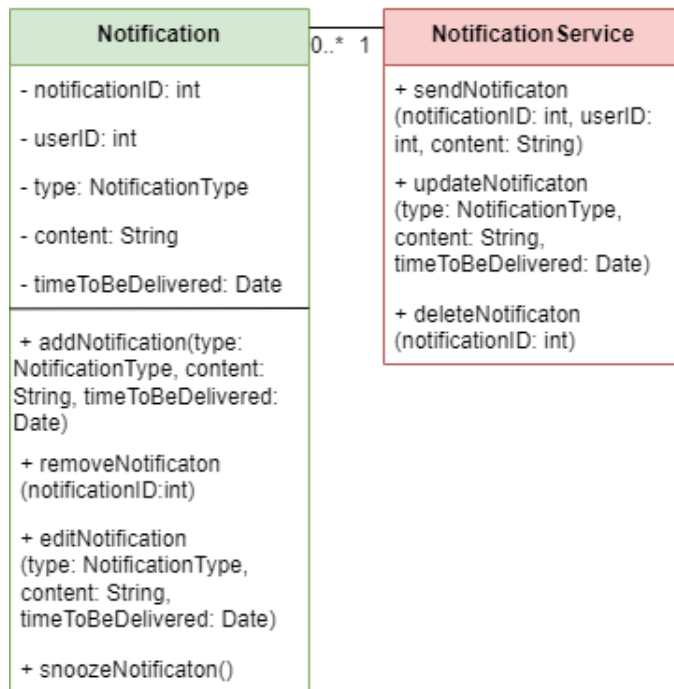
The notification subsystem described in Section 3.2.4 contains 2 classes: `Notification` and `NotificationService`.

The class `Notification` represents a notification with such attributes as `notificationID`, `userID`, `type`, `content`, and `timeToBeDelivered`.

The class `NotificationService` manages requests for sending, updating, and deleting a notification with the respective methods.

**Figure 4**

*The UML Class Diagram of the Notification Package*



Source: Own results.